

Rebelión en la Granja es una aplicación web desarrollada para la gestión de una ganadería. Permite visualizar estadísticas, consultar animales, empleados y actividades, y realizar búsquedas filtradas sobre una base de datos MySQL.

Está construida como parte del Proyecto 3 pero esta parte se enfoca en la parte de Lenguaje de Marcas y Sistemas de Gestión de la Información

Herramientas utilizadas

Capa	Tecnología	¿Por qué la elegimos?
Frontend	HTML5 + CSS3 + JavaScript (ES6)	HTML da la estructura, CSS la apariencia visual y JavaScript la interacción sin tener que recargar la página.
Backend	Node.js + Express	Node.js permite ejecutar código JavaScript en el servidor. Express es un framework que facilita crear rutas y manejar peticiones
Base de datos	MySQL 8 + mysql2	MySQL es el motor relacional heredado del script original de la granja. El paquete mysql2 proporciona un conector para Node.js con soporte para promesas.
Estilos	Bootstrap 5 + CSS personalizado	Bootstrap acelera el desarrollo del borrador de como queremos que se vea la página web usando el sistema de grid responsive y componentes predefinidos. El archivo CSS propio implementa la personalización mediante variables y clases específicas del dominio.
Desarrollo	nodemon	Cada vez que modificamos un archivo del servidor, nodemon lo reinicia automáticamente. Así no tenemos que estar parando y arrancando manualmente el servidor mientras programamos.

Arquitectura y estructura del proyecto



La granja-web sigue una arquitectura simple de servidor único basada en Express. Toda la parte del servidor se encuentra en el archivo llamado `server.js`, aquí se encuentran las distintas apis (get, post), mientras que los archivos estáticos del frontend se muestran directamente desde la carpeta `public/`.

Archivos principales

`server.js` se usa Express, configura los middlewares, declara todas las rutas de la API, gestiona la conexión a MySQL y arranca el servidor en el puerto 3000.

El `package.json` define el nombre del proyecto, la versión, los scripts de arranque (start y dev) y las dependencias express, mysql2, cors, y nodemon para que se actualice el servidor automáticamente si lo estamos desarrollando.

`package-lock.json` es generado automáticamente por npm y fija las versiones exactas de cada dependencia para las instalaciones. Por último, `README.md` contiene la guía de instalación y uso dirigida a cualquier persona que quiera ejecutar el proyecto desde cero, incluyendo requisitos de entorno y configuración de la base de datos.

Carpeta public/

`index.html` es la interfaz gráfica principal, organiza el contenido en tres secciones (Inicio, Información y Consultas) que se muestran y ocultan sin recargar la página. `css/style.css` contiene todo el estilo visual personalizado con variables CSS de color verde y terracota,

tarjetas de estadísticas, badges de salud y layout responsive. `js/main.js` gestiona la navegación entre secciones, la carga asíncrona de estadísticas y tablas mediante la Fetch API, la validación del formulario de consultas y el renderizado dinámico de resultados. Finalmente, `404.html` es la página de error personalizada que se muestra cuando el usuario accede a una ruta HTML inexistente.

Logica interfaz grafica

Navegación entre vistas

La función `navigate(seccion)` oculta las secciones, muestra la elegida añadiendo la clase `active` y actualiza el botón de navegación activo. El bloque hero solo se visualiza en la sección Inicio. Cuando se navega a Información, se fuerza la recarga de la pestaña de animales para que los datos estén siempre actualizados.

```
function navigate(seccion) {  
  // Ocultamos todas las secciones  
  Object.values(SECCIONES).forEach(id => {  
    document.getElementById(id).classList.remove('active');  
  });  
  
  // Mostramos la sección que hemos elegido  
  document.getElementById(SECCIONES[seccion]).classList.add('active');  
  
  // Marcamos el botón activo en el menú  
  // Recorremos los botones y comparamos su posición  
  BOTONES_NAV.forEach((boton, indice) => {  
    const secciones = ['inicio', 'informacion', 'consultas'];  
    // Si coincide, añadimos la clase 'active', si no, la quitamos  
    boton.classList.toggle('active', secciones[indice] === seccion);  
  });  
  
  // El bloque hero solo se muestra en la vista "Inicio"  
  const hero = document.getElementById('hero-block');  
  if (seccion === 'inicio') {  
    hero.style.display = ''; // se muestra  
  } else {  
    hero.style.display = 'none'; // se oculta  
  }  
  
  // Si vamos a "Información", reseteamos la pestaña para recargarla  
  if (seccion === 'informacion') {  
    pestañaActual = ''; // fuerza recarga aunque ya estuviera activa  
    cargarPestaña('animales');  
  }  
}
```

Carga de estadísticas

`cargarEstadisticas()` realiza un fetch a `/api/stats`. Si la respuesta es exitosa, rellena las cuatro tarjetas de la sección Inicio con los valores recibidos. En modo demo añade un badge visual de aviso. En caso de error de red muestra el texto "Error" en cada tarjeta.

```
// ESTADÍSTICAS
async function cargarEstadísticas() {
  try {
    // Hacemos una petición a la API del servidor que nos devuelve los totales
    const respuesta = await fetch('/api/stats');
    if (!respuesta.ok) throw new Error('El servidor no respondió bien');

    // Convertimos la respuesta en un objeto JavaScript
    const resultado = await respuesta.json();

    // Ponemos los números en cada tarjeta
    document.getElementById('stat-animales').innerHTML = resultado.data.animales + (resultado.demo ? '<span class="demo-badge">DEMO</span>' : '');
    document.getElementById('stat-sanos').innerHTML = resultado.data.sanos;
    document.getElementById('stat-empleados').innerHTML = resultado.data.empleados;
    document.getElementById('stat-actividades').innerHTML = resultado.data.actividades;
  } catch (error) {
    // Si hay un error (por ejemplo no hay conexión), mostramos "Error" en todas las tarjetas
    console.error('Error al cargar estadísticas:', error);
    ['stat-animales', 'stat-sanos', 'stat-empleados', 'stat-actividades'].forEach(id => {
      document.getElementById(id).innerHTML = 'Error';
    });
  }
}
```

Pestañas de información

cargarPestaña(tipo) evita peticiones duplicadas comprobando la variable de pestañaActual. Si el tipo es diferente al actual, muestra un spinner de carga, llama a /api/{tipo} y pasa los datos a generarTabla(), que construye las tablas con las columnas para cada entidad en el HTML.

```
async function cargarPestaña(tipo) { // tipo puede ser 'animales', 'empleados', 'actividades'
  // Si ya estamos mostrando esa pestaña, no hacemos nada
  if (pestañaActual === tipo) return;
  pestañaActual = tipo;

  // Activamos visualmente el botón correcto
  const botonesPestaña = document.querySelectorAll('.btn-tab');
  botonesPestaña.forEach((boton, indice) => {
    const pestañas = ['animales', 'empleados', 'actividades'];
    boton.classList.toggle('active', pestañas[indice] === tipo);
  });

  // Mostramos un mensaje de "cargando..."
  const contenedor = document.getElementById('table-container');
  contenedor.innerHTML = '<div class="loader"><div class="spinner-border text-success mb-3"></div><br>Cargando ' + tipo + '...</div>';

  try {
    // Pedimos los datos al servidor
    const respuesta = await fetch('/api/' + tipo);
    const resultado = await respuesta.json();
    // Generamos la tabla con los datos recibidos
    contenedor.innerHTML = generarTabla(tipo, resultado.data, resultado.demo);
  } catch (error) {
    contenedor.innerHTML = '<div class="loader"> Error al cargar los datos.</div>';
  }
}
```

Formulario de consultas filtradas

El usuario selecciona el tipo de filtro desde un <select> y escribe un valor. La función actualizarPlaceholder() adapta el campo de texto: cuando se elige filtro por fecha muestra un input type="date"; en el resto, un campo de texto con ejemplos. Antes del envío, validarFormulario() comprueba que el tipo esté seleccionado y que el filtro no contenga caracteres raros (" , , , , < , > , 1).

```
// FORMULARIO DE CONSULTAS

// Cambia el texto de ejemplo cuando se elige un tipo de consulta
function actualizarPlaceholder() {
  const tipo = document.getElementById('q-tipo').value;
  const inputFiltro = document.getElementById('q-filtro');

  const ejemplos = {
    animales_especie: 'Ej: Bovino, Porcino, Ovino...',
    empleados_rol: 'Ej: Veterinario, Peón, Encargado...',
    actividades_fecha: 'Ej: 2024-05-01',
    animales_salud: 'Ej: Sano, En tratamiento...',
  };

  inputFiltro.placeholder = ejemplos[tipo] || 'Escribe un filtro...';

  // Si es consulta por fecha, mostramos un selector de fecha; si no, un campo de texto
  if (tipo === 'actividades_fecha') {
    inputFiltro.type = 'date';
  } else {
    inputFiltro.type = 'text';
  }

  // Limpiamos los mensajes de error anteriores
  limpiarErrores();
}


```

Si es válido, ejecutarConsulta() envía la petición POST y renderiza los resultados con un encabezado que indica la consulta realizada y el número de registros encontrados.

```
// Envía la consulta al servidor y muestra los resultados
async function ejecutarConsulta() {
  if (!validarFormulario()) return; // si no es válido, paramos aquí

  const tipo = document.getElementById('q-tipo').value;
  const filtro = document.getElementById('q-filtro').value.trim();
  const boton = document.getElementById('btn-consultar');
  const zonaResultados = document.getElementById('query-results');

  // Desactivamos el botón mientras se busca y ponemos texto de "Buscando..."
  boton.disabled = true;
  boton.innerHTML = `<span class="spinner-border spinner-border-sm me-2"></span>Buscando...`;

  // Quitamos resultados anteriores
  zonaResultados.style.display = 'none';

  try {
    const respuesta = await fetch('/api/consulta', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ tipo: tipo, filtro: filtro })
    });

    const resultado = await respuesta.json();

    if (!respuesta.ok) {
      // Si el servidor devuelve un error, mostramos el mensaje
      zonaResultados.innerHTML = `<div class="loader"> ${resultado.error}</div>`;
    } else {
      // Averiguamos de qué tabla vienen los datos (animales, empleados o actividades)
      const tabla = tipo.split('_')[0];
      let avisoDemo = '';
      if (resultado.demo) {
        avisoDemo = `<p style="color:var(--rust); font-size:0.8rem; margin-bottom:0.5rem;>⚠ Datos de demostración</p>`;
      }

      // Construimos el HTML del encabezado de resultados « la tabla
      zonaResultados.innerHTML = `
      <div class="results">
        <div class="results-header">
          <div class="results-titulo">Resultados: ${resultado.consulta}</div>
          <div class="results-count">${resultado.total} registros</div>
        </div>
        <div>
          <table>
            <thead>
              <tr>
                <th>${resultado.tabla}</th>
              </tr>
            </thead>
            <tbody>
              <tr>
                <td>${resultado.data[0]}</td>
              </tr>
            </tbody>
          </table>
        </div>
      </div>
      `;

      zonaResultados.style.display = 'block';
    }
  } catch (error) {
    // Error de conexión (no se pudo llegar al servidor)
    zonaResultados.innerHTML = `<div class="loader"> Error de conexión con el servidor.</div>`;
    zonaResultados.style.display = 'block';
  } finally {
    // Activamos de nuevo el botón, tanto si fue bien como si hubo error
    boton.disabled = false;
    boton.innerHTML = `<i class="bi bi-search me-1"></i> Buscar`;
  }
}


```

Función crearBadgeSalud()

Genera un `<span>` con clase de color según el estado de salud del animal: verde para "sano", amarillo para "en tratamiento" y rojo para cualquier otro estado. Esto permite identificar visualmente el estado de cada animal en la tabla de información.

```
// BADGE DE COLOR SEGÚN EL ESTADO DE SALUD
function crearBadgeSalud(estado) {
  if (!estado) return '-'; // por si viene vacío

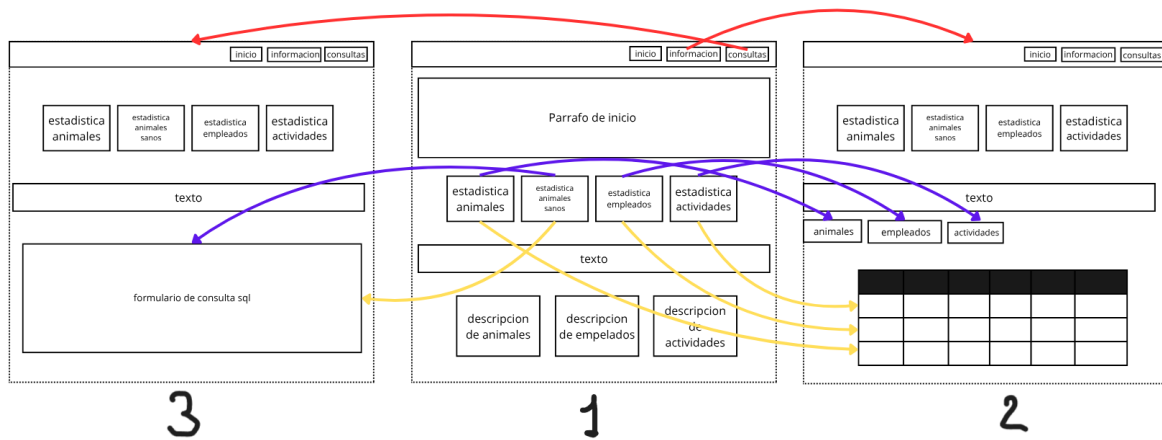
  const texto = estado.toLowerCase();

  if (texto.includes('sano')) {
    return `<span class="badge-salud verde">${estado}</span>`;
  } else if (texto.includes('tratamiento')) {
    return `<span class="badge-salud ambar">${estado}</span>`;
  } else {
    // Cualquier otro estado (ej. enfermo) lo mostramos en rojo
    return `<span class="badge-salud rojo">${estado}</span>`;
  }
}
```

```
let columnas = [];
let filasHTML = '';

// Dependiendo del tipo, creamos las cabeceras y las filas
if (tipo === 'animales') {
  columnas = ['ID', 'Especie', 'Raza', 'Nacimiento', 'Identificador', 'Salud', 'Ubicación'];
  filasHTML = datos.map(animales => `
    <tr>
      <td><code>${animales.id}</code></td>
      <td>${animales.especie}</td>
      <td>${animales.raza}</td>
      <td>${animales.fecha_nacimiento || '-'}</td>
      <td><code>${animales.identificador}</code></td>
      <td>${crearBadgeSalud(animales.estado_salud)}</td>
      <td>${animales.ubicacion}</td>
    </tr>`.join(''));
}
```

Diagrama de vistas



Logica del servidor

inicio

Al arrancar, Express configura tres middlewares globales:

```
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static('public'));
```

`express.json()` para parsear cuerpos JSON en peticiones POST, `express.urlencoded()` para formularios y `express.static('public')` para mostrar los archivos del frontend directamente.

Conexion a Mysql

La conexión se crea con `mysql2.createConnection()` apuntando a `localhost`, usuario y la base de datos se configura en el archivo. Si la base de datos MySQL no está disponible, se muestra un aviso por consola que nos notifica si se conecto la base de datos tambien se muestran todas las rutas y si no se llega a conectar a mysql devuelve datos de demostración (DEMO)

Función `hacerConsulta()`

Recibe la sentencia sql, los parámetros y un callback. Si se produce cualquier error de lo captura, lo registra en consola y llama al callback con null, provocando que la ruta vaya a los datos de demo automáticamente.


```
// FUNCIÓN PARA HACER CONSULTAS
function hacerConsulta(sql, params, callback) {
  conexion.query(sql, params, (error, resultados) => {
    if (error) {
      console.log('Error en la consulta:', error.message);
      callback(null);
    } else {
      callback(resultados);
    }
  });
}
```

Apis

1. /api/animales GET : devuelve todos los registros de la tabla animales

```
// APIs GET
app.get('/api/animales', (req, res) => {
  hacerConsulta('SELECT * FROM animales', [], (datos) => {
    res.json({
      data: datos || DEMO.animales,
      demo: !datos
    });
  });
});
```

2. /api/empleados GET : devuelve todos los registros de la tabla empleados

```
app.get('/api/empleados', (req, res) => {
  hacerConsulta('SELECT * FROM empleados', [], (datos) => {
    res.json({
      data: datos || DEMO.empleados,
      demo: !datos
    });
  });
});
```

3. /api/actividades GET : devuelve los registro de la vista vista_actividades las cuales une a las actividades con los empleados y animales.

```
app.get('/api/actividades', (req, res) => {
  hacerConsulta('SELECT * FROM vista_actividades', [], (datos) => {
    res.json({
      data: datos || DEMO.actividades,
      demo: !datos
    });
  });
});
```

4. /api/stats GET: Retorna cuatro contadores o estadísticas siendo el total de animales, animales sanos, empleados y actividades. Si hay conexión activa ejecuta cuatro COUNT(*) en paralelo con Promise.all(); si no, usa los arrays de DEMO.


```
// NUEVA RUTA: GET /api/stats
app.get('/api/stats', (req, res) => {
  // Si hay conexión a la BD, hacemos consultas reales
  if (conexion.state === 'authenticated') {
    const sqlAnimales = 'SELECT COUNT(*) AS total FROM animales';
    const sqlSanos = 'SELECT COUNT(*) AS total FROM animales WHERE estado_salud = 'Sano'';
    const sqlEmpleados = 'SELECT COUNT(*) AS total FROM empleados';
    const sqlActividades = 'SELECT COUNT(*) AS total FROM actividades';

    Promise.all([
      new Promise((resolve) => hacerConsulta(sqlAnimales, [], resolve)),
      new Promise((resolve) => hacerConsulta(sqlSanos, [], resolve)),
      new Promise((resolve) => hacerConsulta(sqlEmpleados, [], resolve)),
      new Promise((resolve) => hacerConsulta(sqlActividades, [], resolve))
    ]).then(([ani, san, emp, act]) => {
      res.json({
        data: {
          animales: ani ? ani[0].total : DEMO.animales.length,
          sanos: san ? san[0].total : DEMO.animales.filter(a => a.estado_salud === 'Sano').length,
          empleados: emp ? emp[0].total : DEMO.empleados.length,
          actividades: act ? act[0].total : DEMO.actividades.length
        },
        demo: !ani
      });
    });
  } else {
    // Sin BD: datos de demostración
    const totalSanos = DEMO.animales.filter(a => a.estado_salud === 'Sano').length;
    res.json({
      data: {
        animales: DEMO.animales.length,
        sanos: totalSanos,
        empleados: DEMO.empleados.length,
        actividades: DEMO.actividades.length
      },
      demo: true
    });
  }
});
});
```

5. /api/consulta POST: El cuerpo JSON debe incluir el tipo (animales_especie, empleados_rol, actividades_fecha o animales_salud) y filtro. Devuelve los registros coincidentes, el total y la consulta realizada.

```
// API POST
app.post('/api/consulta', (req, res) => {
  const tipo = req.body.tipo;
  const filtro = req.body.filtro;

  if (!tipo) {
    return res.status(400).json({
      error: 'Falta el campo tipo',
      ejemplo: '{ "tipo": "animales_especie", "filtro": "Bovino" }'
    });
  }
});
```

Funcionamiento de la pagina web

Cuando el usuario abre el navegador e introduce la ruta del servidor, este solicita index.html (la interfaz visual) a Express, que lo muestra desde la carpeta public/. Al cargar la página, main.js llama automáticamente a la función cargarEstadisticas(), que realiza

una petición fetch a /api/stats. El servidor consulta la base de datos MySQL, o si no tenemos la base de datos configurada usa el DEMO , y devuelve los cuatro contadores de estadísticas en JSON para rellenar las tarjetas ubicadas en la sección Inicio.

Si el usuario se dirige a la sección Información, se carga la Pestaña ("animales"), que hace fetch a la /api/animales. El servidor ejecuta la consulta sql `SELECT * FROM animales` que devuelve todos los registros; el index.html los recibe y muestra la tabla HTML dinámicamente con la funcion `generar Tabla()`. Si el usuario cambia la pestaña a Empleados o Actividades, el index.html realiza nuevas peticiones solo cuando la pestaña seleccionada es diferente a la actual.

En la sección Consultas, el usuario elige un tipo en el desplegable y escribe un valor. Tras pasar la validación del formulario, el metodo `ejecutarConsulta()` envía un POST a /api/consulta con el tipo y el filtro. El servidor ejecuta la consulta SQL y devuelve los registros coincidentes; luego se muestra la tabla de resultados acompañada de un encabezado con el nombre de la consulta y el número total de registros encontrados.